

Transferleistung Theorie/Praxis* 4

Matrikelnummer:	7410
Freigegebenes Thema:	Untersuchung von Schema Evolution verschiedener Datenbanktypen für Business Process Management Systeme
Studiengang, Zenturie:	A16a

** Studierende, die unter den Anwendungsbereich der PVO bis 03.02.2015 fallen, fertigen Transferleistungen weiterhin in der Form von Praxisberichten an und der Begriff hält Einzug in das Abschlusszeugnis. Ab dem Jahrgang 2016 hat der Begriff vollumfängliche Gültigkeit. In der Kommunikation hält der Begriff Transferleistungen ab sofort Einzug.*

Inhaltsverzeichnis

Tabellenverzeichnis	ii
Abbildungsverzeichnis	iii
Abkürzungsverzeichnis	iv
1 Einleitung	1
2 Grundlagen	2
2.1 Schema Evolution	2
2.2 Workflow Engines	2
2.3 Relationale Datenbanken	2
2.4 Graph Datenbanken	3
3 Vergleichbare Arbeiten	5
4 Analyse	5
4.1 Methodik	5
4.2 Fallbeispiel	6
4.3 Objektive Messungen	7
4.4 Subjektive Messungen	8
4.5 Diskussion	9
5 Fazit	9
Quellenverzeichnis	iv
Anhang	vi

Tabellenverzeichnis

1	Ausführungsdauer in Millisekunden (Dauer; Standardabweichung)	7
2	Größe im Arbeitsspeicher in Kilobyte (KB)	7
3	Physikalische Größe der Datenbank in KB	7
4	Benötigte Zeilen Skript und Anzahl der Statements in Structured Query Language (SQL)/Cypher	8

Abbildungsverzeichnis

1	Beispielhafte Darstellung eines Graphen von Personen [14]	4
2	Der Workflow eines Lagersystems mit Datenbankattributen	6

Abkürzungsverzeichnis

API	Application Programming Interface
BPM	Business Process Management
BPMN	Business Process Management Notation
BPMS	Business Process Management System
DCL	Data Control Language
DDL	Data Definition Language
DML	Data Manipulation Language
KB	Kilobyte
LOC	Lines of Code
LPG	Labeled Property Graph
SQL	Structured Query Language

1 Einleitung

Moderne Business Process Management (BPM) Systeme sind zur Erstellung und Verbesserung von Workflows unerlässlich in Großunternehmen geworden. Sie ermöglichen die Identifikation von optimierbaren Prozessen, die durch Methoden wie Automatisierung erfolgreicher und schneller bearbeitet werden können.[1] Dabei kann zusätzlich eingesetzte Software die Mitarbeiter unterstützen oder für einen Teil des Workflows vollkommen ersetzen.[2]

BPM Systeme sind selbst schon lange durch Software-Systeme gestützt. Standards wie die Business Process Management Notation (BPMN) ermöglichen eine einfache, maschinenlesbare Beschreibung von Business Prozessen auf Basis einer grafischen Spezifikationsprache.[3] Doch nicht nur während der Erstellung werden Software-Systeme eingesetzt. Auch während der Laufzeit spielen die Workflow Engines eine entscheidende Rolle, wobei sie eine Vielzahl von Tätigkeiten übernehmen. Software gestützte Workflows protokollieren jeden Arbeitsschritt und stellen Schnittstellen bereit, um mit diesen zu interagieren.[4]

Damit lässt sich eine Historie zur lückenlosen Nachverfolgung eines Arbeitsprozesses erstellen. Die Persistierung übernimmt eine Datenbank, welche diverse Eigenschaften besitzen muss, um die sich ändernden Anforderungen eines Workflowsystems zu erfüllen. Jederzeit können Prozesse verändert, gelöscht oder hinzugefügt werden. Aus diesem Grund muss die verwendete Datenbank *Schema Evolution* unterstützen.¹[6] Insofern stellt sich folgende Frage: Welcher Datenbanktyp eignet sich am besten für Business Process Management System (BPMS) angesichts der Unterstützung für Schema Evolution?

Die Unterstützung verschiedener Datenbanktypen für Schema Evolution zu untersuchen ist Ziel dieser Arbeit. Zunächst werden dafür die Grundlagen erläutert. Dabei wird näher auf die Tätigkeiten einer Workflow Engine und die dabei anfallenden Daten eingegangen. Zusätzlich werden die grundlegenden Eigenschaften von relationalen und graphbasierten Datenbanken dargestellt. Die Unterschiede in der Funktionsweise der Schema Evolution werden hervorgehoben. Anschließend werden ähnliche wissenschaftliche Arbeiten zu diesem Thema betrachtet. Um daraufhin die verschiedenen Datenbanktypen bewerten zu können, wird ein Fallbeispiel entwickelt, welches verschiedene Aspekte der Schema Evolution abdeckt. Anschließend wird dieses mit je einem Vertreter eines Datenbanktyps durchgeführt, protokolliert und ausgewertet. Zuletzt wird auf Basis dieser Ergebnisse ein Fazit erstellt, womit eine Empfehlung für den zu verwendenden Datenbanktypen in einem BPMS abgegeben wird.

¹Schema Evolution ist die Möglichkeit, Daten und Schema trotz Veränderungen des Schemas konsistent zu halten.[5]

2 Grundlagen

2.1 Schema Evolution

In den meisten Software-Systemen werden Datenbanken zur Persistierung der internen Datenstrukturen verwendet. Da bei andauernder Entwicklung diese diversen Änderungen unterliegen, ist eine zwangsläufige Folge auch eine Änderung am Schema der verwendeten Datenbank.[5] Diese fortwährende Änderung wird als Schema Evolution bezeichnet. Um bereits bestehende Daten innerhalb der Datenbank an die Änderungen des Schemas anzupassen, kommen dabei meist verschiedene Datenbankmechanismen und externe Tools zum Einsatz.[7]

2.2 Workflow Engines

Die Tätigkeiten einer Workflow-Engine sind vielfältig. Um diese zu konkretisieren, wird die quelloffene Workflow-Engine *Camunda BPM* exemplarisch betrachtet. Camunda kann als Java-Software entweder eigenständig als Server arbeiten oder in eine andere Applikation eingebettet werden.[8] Folgendes sind Standardtätigkeiten bei der Verwendung von Camunda:

- Prozesse können programmatisch oder durch ein Benutzerinterface erstellt und bearbeitet werden.
- Automatismen können nach Erreichen eines bestimmten Arbeitsschrittes gestartet werden und sich nach Fertigstellung an der Workflow Engine zurückmelden.
- Auf Basis des aktuellen Zustands eines Workflows können automatisch Entscheidungen zum weiteren Fortfahren getroffen werden.
- Historisierung aller vergangenen Arbeitsprozesse.

Zusätzlich ermöglicht Camunda über eigene Historisierungs-Event-Handler jegliche historisierten Prozessschritte zur eigenen Nutzung zu verwenden. Dies erlaubt Transformation und Persistierung der Daten in einer dezidierten Datenbank. Für den operativen Gebrauch greift Camunda auf eine relationale Datenbank zurück.[9]

Theoretisch können alle mit dem Workflow-Prozess zusammenhängenden Daten in einer Datenbank persistiert werden. Im Zuge der Schema Evolution sind allerdings ausschließlich Strukturen relevant, die sich durch Anpassungen im Workflow verändern. Daher werden nur diese bei der späteren Analyse in Betracht gezogen.

2.3 Relationale Datenbanken

Relationale Datenbanken sind die verbreitetsten Datenbanken auf dem Markt.[6] Aufbauend auf der relationalen Algebra stellt solch eine Datenbank ihre Daten in einer Relation bzw. Tabelle dar. Eine Relation besteht aus mehreren Attributen, welche jeweils als eine Spalte einer Tabelle angesehen werden können. Ein Datensatz einer relationalen Datenbank besteht aus den Daten

einer Zeile der Tabelle. Dies lässt sich als n -Tupel beschreiben, wobei n die Anzahl der Attribute in der Relation ist. Alle Relationen einer Datenbank werden als Schema zusammengefasst.[10]

Dabei hat sich die relationale Datenbank nicht nur an den Konstrukten der relationalen Algebra orientiert, sondern auch an deren Operationen. Allen voran kommen Restriktion und Projektion, welche zur Filterung von Daten zuständig sind. Die Projektion kann zusätzlich dazu genutzt werden neue Daten zu berechnen.[11] Eine weitere wichtige Operation ist die Verbundoperation (engl. *join*). Sie ermöglicht die Bildung einer Tabelle aus zwei oder mehr Ursprungstabellen auf Basis eines gemeinsamen Verbundkriteriums.[10][11] Moderne relationale Datenbanken bieten viele weitere Operationen und Funktionalitäten an, welche allerdings zur Beantwortung der Forschungsfrage zu vernachlässigen sind.

Ein üblicher Bestandteil der Arbeit mit relationalen Datenbanken ist die Structured Query Language (SQL). Anstatt eines Application Programming Interface (API) stellen relationale Datenbanken in der Regel eine Text-Schnittstelle bereit, damit Programme mit ihnen interagieren können. SQL erfüllt dabei gleich mehrere Funktionen: Neben der Abfrage von Datenbankelementen (Data Control Language (DCL)) erlaubt es zudem die Datenmanipulation (Data Manipulation Language (DML)) und die Anpassung des Schemas (Data Definition Language (DDL)).[12] Unter der Datenmanipulation ist die Erstellung, Veränderung und Löschung von Daten zu verstehen.

Im Hinblick auf die Schema Evolution ist der Aspekt der Schemaanpassung von besonderer Bedeutung. Mithilfe der DDL können neue Datenbankstrukturen erstellt, angepasst und gelöscht werden. Damit können Relationen und alle verbundenen Strukturen durch SQL bearbeitet werden.[12] Um die Anforderungen der Schema Evolution zu erfüllen, sind jedoch alle Teile von SQL zu verwenden. Zur Veränderung des Schemas sind die Funktionen der DDL ausreichend, allerdings betrifft die Schema Evolution ebenfalls die Daten einer Tabelle. Zum Abfragen bestehender Daten und ihre Manipulation muss jeweils die DCL und DML ebenfalls genutzt werden.

Eine bestehende Problematik der Schema Evolution in relationalen Datenbanken sind DCL-Skripte, die von Anwendungen genutzt werden, um Daten abzufragen. Da diese ebenfalls wegen des veränderten Schemas angepasst werden müssen, kann es zu einer großen Menge an weiteren Wartungsarbeiten kommen. Im Falle einer Schema Evolution in der MediaWiki² Software müssen pro Schemaänderung bis zu 70% der DCL-Skripte angepasst werden.[13]

2.4 Graph Datenbanken

Ähnlich wie relationale Datenbanken bauen Graph Datenbanken auf einem mathematischen Modell auf. Die dafür benutzten Graphen sind definiert als $G(N, E)$, wobei N für die Menge der Knoten (Nodes) und E für die Menge der Kanten (Edges) steht, die die Knoten miteinander verbinden. Jeder Knoten besitzt eine Menge von Eigenschaften, weshalb solch ein Graph auch beschrifteter Eigenschaften Graph (Labeled Property Graph (LPG)) genannt wird. Ähnlich wie

²Die MediaWiki Software ist ein Content Management System zur Erstellung und Wartung von Wikis

bereits ein Datenelement in einer relationalen Datenbank besitzt ein Knoten ein n-Tupel von Eigenschaften, welche jeweils aus einem Schlüssel-Werte-Paar bestehen. Die Knoten verbindenden Kanten des Graphen stellen die Beziehungen zwischen den Datenelementen dar und sind im Allgemeinen benannt und gerichtet. Dabei können Kanten, so wie Knoten auch, Eigenschaften beinhalten.[14]

Im Vergleich zu relationalen Datenbanken ist eine Graph Datenbank darauf ausgelegt die Beziehungen zwischen verschiedenen Datenelementen zu modellieren. Während eine relationale Datenbank Beziehungen durch Fremdschlüssel lediglich fachlich inferieren lässt, sind solche Beziehungen in Graph Datenbanken essentieller Bestandteil der Datenbank. Dabei werden die Beziehungen meist als physikalische Informationen genauso wie die Knoten im Speicher festgehalten. Hinsichtlich der Leistung der Datenbank hat das zur Folge, dass das Abfragen von Daten und ihrer Beziehungen effizienter verarbeitet werden können als in relationalen Datenbanken.[14]

Bei der folgenden Betrachtung von Interaktion mit Graph Datenbanken wird exemplarisch Neo4j verwendet. Neo4j ist seit mehreren Jahren bereits (Stand 09.2019) die relevanteste Graph Datenbank, verglichen mit den anderen auf dem Markt erhältlichen Systemen [15]. Dabei enthält es Funktionalitäten, die es für die Verwendung in dieser Arbeit besonders attraktiv machen.

Zunächst implementiert Neo4j eine eigene Abfragesprache namens Cypher. Mit dieser lassen sich Abfrage und Manipulation der Datenbank tätigen ohne auf eine Programmiersprachen spezifische API angewiesen zu sein. Cypher ermöglicht es, die mathematischen Graph-Strukturen in einer kompakten Abfragesprache darzustellen.[14] In Abbildung 1 wird ein Graph gezeigt, in dem Bekanntschaftsbeziehungen von mehreren Personen zueinander ausgedrückt werden. Die Knoten repräsentieren jeweils eine Person mit der Eigenschaft `name`, während die Kanten eine Bekanntheitsbeziehung ausdrücken mit dem Label `KNOWS`. Dieser Graph lässt sich in Cypher folgendermaßen in Textform abbilden:

```
(emil) <-[:KNOWS]-(jim)-[:KNOWS]->(ian)-[:KNOWS]->(emil)
```

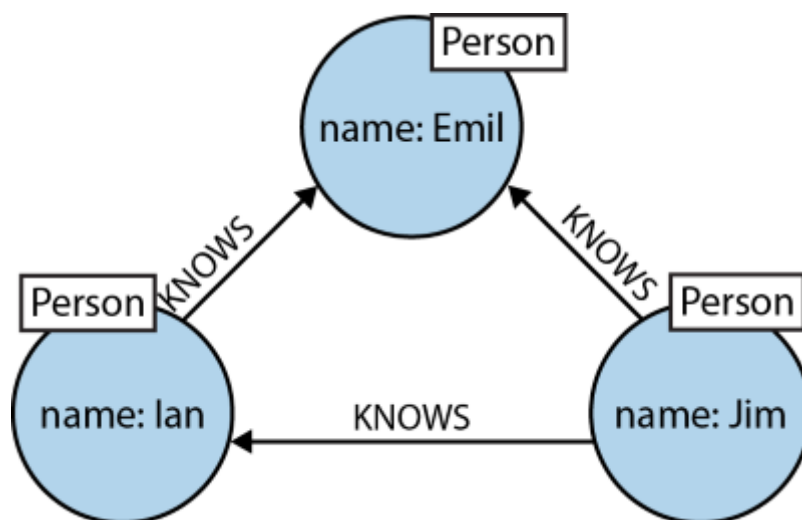


Abbildung 1: Beispielhafte Darstellung eines Graphen von Personen [14]

Des Weiteren besitzt Neo4j die Möglichkeit über Indizes und Constraints ein implizites Schema zu definieren. Der Begriff Constraint ist in diesem Kontext anders zu verstehen als bei relationalen Datenbanken, welche jene nutzen um Attribute von Relationen mit Metainformationen zu versehen. Stattdessen können mithilfe von Constraints in Neo4j überhaupt erst Eigenschaften und deren Typ im Schema angelegt werden. Dieser Punkt unterscheidet Neo4j von anderen Graph Datenbanken hinsichtlich der Schema Evolution, da Neo4j seine Daten gegen das angegebene Schema validiert. Die Validierung ist hilfreich zu einer effektiven Schema Evolution, weil damit sichergestellt wird, dass die Daten zum Ende des Prozesses die gewünschte Struktur besitzen. Das Schema wird mit einer DDL ähnlichen Sprache mit Cypher angelegt.[16]

3 Vergleichbare Arbeiten

Es gibt viele Vergleiche von relationalen Datenbanken zu Graph Datenbanken. Insbesondere die Abfragegeschwindigkeit in verschiedenen Szenarien wird in Arbeiten wie [17] oder [18] ausgiebig untersucht. Dabei fällt besonders auf, dass traversierende Abfragen – wie die Abbildung einer Graphenstruktur – mit einer Graph-Datenbank üblicherweise um eine Größenordnung schneller fertiggestellt werden als in einer relationalen Datenbank. [17] Ebenfalls scheint eine Graph Datenbank eine bessere Skalierung bezüglich der Anzahl der abgefragten Datenelemente zu besitzen [18].

Zusätzlich werden neben der Geschwindigkeit noch andere Metriken analysiert. Meist spielt die physikalische Größe der Datenbank eine große Rolle. Wie in [17] zu sehen ist, sind Neo4j Datenbanken in Szenarien, in denen viele Integer verwendet werden, größer als die damit verglichene MySQL Datenbank. Ebenso werden nicht messbare Faktoren in den Vergleich der Datenbanktypen miteinbezogen. In [17] und [19] wird die fehlende Einheitlichkeit sowie der unzureichende Reifegrad der Software kritisiert.

4 Analyse

4.1 Methodik

Um die Frage zu beantworten, welcher der beiden vorgestellten Datenbanktypen sich hinsichtlich Schema Evolution von BPMS besser eignet, wird ein Fallbeispiel bearbeitet. Dieses soll möglichst viele Aspekte der Schema Evolution in BPMS abbilden. Die Bearbeitung erfolgt anschließend durch jeweils einen Repräsentanten jedes Datenbanktyps. Als relationale Datenbank wird SQLite gewählt, da es kein komplexes Datenbankmanagementsystem benötigt und alle notwendigen SQL Statements für Schema Evolution implementiert [20]. Das zuvor vorgestellte Neo4j (Version 3.5.9) wird für diesen Zweck als Graph Datenbank verwendet. Alle Schritte der Schema Evolution werden in SQL bzw. Cypher in Textform (Skript) festgehalten. (siehe

Anhang S. vi - vii) Die erstellten Skripte werden anschließend unter Betrachtung verschiedener Aspekte gemessen. Dabei lassen sich die im folgenden vorgestellten Metriken anwenden.

Zunächst werden objektive Messungen betrachtet. Darunter fallen alle Messungen, die während der Laufzeit der Skripte erfasst werden können. Das beinhaltet Laufzeit (Performance), Arbeitsspeicherverbrauch, sowie Festplattenspeicherverbrauch. Um eine höhere Genauigkeit bei der Ausführung des Beispiels zu erreichen, wird es mehrmals durchgeführt, wobei das Ergebnis als Tupel von arithmetischen Mittel der Messungen sowie der Standardabweichung angegeben wird. Zudem können direkt Metriken aus den Skripten analysiert werden. Dafür lassen sich Lines of Code (LOC) oder die Anzahl der Statements verwenden. Auch wenn diese beiden Metriken sich nicht explizit auf die Leistung der Datenbank auswirken, so sind sie dennoch relevant für die Benutzbarkeit der Datenbank.[17]

Außerdem lassen sich neben den objektiven auch subjektive Messungen in Betracht ziehen. Diese Aspekte der Datenbank sind nicht quantifizierbar, sind jedoch essentiell zur Beantwortung der Frage, wie leicht eine Schema Evolution auf einer Datenbank durchzuführen ist. Allem voran ist die Verwendung der genutzten Sprache zu bewerten. Dabei gilt, umso einfacher die Statements zu verfassen sind, desto besser ist die Sprache für eine Schema Evolution geeignet.[17]

4.2 Fallbeispiel

Im Fallbeispiel wird ein simples Lagersystem behandelt. Der dazugehörige Workflow ist in Abbildung 2 zu sehen. Darin werden Waren angenommen, für eine gewisse Dauer eingelagert und anschließend wieder versendet. Dabei werden jeweils das Datum der Annahme und Sendung vermerkt. Erweiterte Möglichkeiten eines BPM wie beispielsweise Entscheidungen werden nicht betrachtet.



Abbildung 2: Der Workflow eines Lagersystems mit Datenbankattributen

SQLite und Neo4j haben zunächst die Aufgabe den Workflow in ihrem Schema zu repräsentieren. Dazu werden 1.000.000 randomisierte Datensätze in jeder Datenbank erzeugt. Anschließend werden vier Szenarien bearbeitet, die mithilfe von SQL und Cypher gelöst werden sollen. Dabei gilt im Allgemeinen, dass alle Szenarien fehlerlos von den Systemen bewältigt werden müssen. Die Szenarien sehen wie folgt aus:

1. Das Workflowsystem wird nicht mehr länger in dem Bereich betrieben. Daher werden nun alle Daten und Schemas gelöscht.

2. Das Lagersystem wird umgebaut zu einem Durchleitungsposten. Anstatt Waren, bevor sie weiter gesendet werden, zwischenzulagern, werden sie nun direkt verschickt. Daher entfällt der Lagerungsschritt des Workflows. Bestehende Daten der Lagerung müssen gelöscht, sowie der Schritt der Lagerung im Workflow überbrückt werden.
3. Bevor sie wieder versendet werden, müssen die Pakete begutachtet werden. Eine solche Begutachtung kann entweder positiv, neutral oder negativ ausfallen. In beiden Fällen wird das Paket verschickt. Dafür wird ein neuer Schritt im Workflow angelegt. Bereits historisierte Ereignisse müssen angepasst werden, sodass sie eine neutrale Begutachtung erhalten.
4. Um das Lagersystem übersichtlich zu halten, soll nun im Lagerungsschritt das Stockwerk gespeichert werden, in dem das Paket liegt. Bestehende Lagerungsschritte werden angepasst, sodass das Paket im Erdgeschoss zu finden ist.

4.3 Objektive Messungen

Die objektiven Messungen werden im Folgenden tabellarisch dargestellt:

Tabelle 1: Ausführungsdauer in Millisekunden (Dauer; Standardabweichung)

Datenbank	Szenario 1	Szenario 2	Szenario 3	Szenario 4
SQLite	3342; 120	19532; 891	21930; 964	3; <1
Neo4j	4580; 430	7736; 693	9356; 731	1074; 32

Tabelle 2: Größe im Arbeitsspeicher in KB

Datenbank	Initial	Szenario 1	Szenario 2	Szenario 3	Szenario 4
SQLite	0	187.318	261.919	286.361	2.601
Neo4j	1.326.421	493.173	1.298.117	1.673.183	1.398.885

Tabelle 3: Physikalische Größe der Datenbank in KB

Datenbank	Initial	Szenario 1	Szenario 2	Szenario 3	Szenario 4
SQLite	715.261	4	517.381	837.381	715.268
Neo4j	2.395.241	38	1.879.103	2.873.154	2.531.372

Tabelle 4: Benötigte Zeilen Skript und Anzahl der Statements in SQL/Cypher

Datenbank	Initial	Szenario 1	Szenario 2	Szenario 3	Szenario 4
SQLite	16; 3	3; 3	20; 9	27; 10	2; 1
Neo4j	5; 5	7; 6	4; 2	5; 3	4; 2

Wie in Tabelle 1 abzulesen ist, gibt es keine signifikanten Unterschiede in der Laufzeit der Szenarien, abgesehen von Szenario 4. Dennoch konnte Neo4j in Szenario 2 und 3 eine im Durchschnitt 2,3-fach kürzere Zeit aufweisen als SQLite. Lediglich in Szenario 4 war Neo4j um mehrere Größenordnungen langsamer. Das hängt mit der Natur des ausgeführten Skripts zusammen. Während in SQLite lediglich eine Anpassung am Schema vorgenommen wurde, mussten in Neo4j alle Datensätze angepasst werden.

In Tabelle 2 und 3 sind die Größen der einzelnen Szenarien im Arbeitsspeicher und auf der Festplatte zu sehen. In beiden Tests zeigt SQLite eine bessere Leistung. Dabei ist notwendig zu erwähnen, dass Neo4j einen Datenbankserver erstellt, welcher dauerhaft läuft, während SQLite lediglich während einer Transaktion aktiv ist. Insbesondere im Arbeitsspeicherverbrauch zeigt SQLite eine deutlich bessere Leistung als Neo4j, was angesichts von Hardwarelimitierungen relevant ist. Die deutlich höhere physikalische Größe von Neo4j deckt sich mit anderen Arbeiten wie [17].

Die Metriken zu den ausgeführten Skripten sind in Tabelle 4 dargestellt. Besonders auffällig ist dabei, dass in Szenario 2 und 3 in Cypher lediglich ein Drittel der Statements und nur ein Fünftel der Zeilen von SQL benötigt wurden. Cypher wirkt damit mächtiger und kompakter als SQL bei der Abfrage und Manipulation von Daten. In Bezug auf die Definition des Schemas verhalten sich Cypher und SQL ähnlich. Dennoch ist das Schema in Neo4j nicht so einschränkend wie in SQLite. Es können Graphen erstellt werden, die mehr Eigenschaften besitzen als im Schema definiert ist, da es keine Möglichkeit gibt, die Eigenschaften eines Knoten zu limitieren.

4.4 Subjektive Messungen

Bei der Erstellung der Skripte ist schnell aufgefallen, dass Cypher die Möglichkeit besitzt mehrere Aufgaben in einem Statement zu erfüllen. So können in einem einzigen Statement mit `MATCH...CREATE...DELETE...RETURN` Daten abgefragt, neue daraus erstellt, abgefragte Daten gelöscht und zuletzt zurückgegeben werden. Zur Befüllung der Neo4j Datenbank wurde zudem auf eine `FOREACH` Schleife zurückgegriffen. Ebenso erlaubt Cypher Verzweigungen, welche Cypher deutlich mehr zu einer Skriptsprache machen als eine normale Abfragesprache. Da die Syntax SQL ähnelt, fällt es nach einer Einarbeitungsphase nicht schwer damit zu arbeiten. Dennoch unterscheiden sich die Abfragen von (Teil-)Graphen stark von den aus SQL bekannten *Joins* zwischen mehreren Tabellen.

Hinsichtlich der Veränderung des Schemas ermöglicht Cypher jedoch nicht wie SQL in einem Statement ein Gesamtschema für einen Knoten anzulegen. Stattdessen wird jede Eigenschaft einzeln entweder mit einem Index oder Constraint belegt. Damit ist die Unterstützung durch Neo4j für Schemas im Allgemeinen wie zuvor bereits beschrieben nicht so stark ausgeprägt wie in relationalen Datenbanken.

4.5 Diskussion

Im Folgenden wird die Aussagekräftigkeit der Daten behandelt. Dabei wird sich auf das Fallbeispiel und Auffälligkeiten der Messungen bezogen.

Zunächst stellt das Fallbeispiel nur einen bedingt realistischen Fall dar. Workflows enthalten häufig Entscheidungsmöglichkeiten, durch die das Datenmodell komplexer wird. Unter Umständen könnte einer der beiden Datenbanktypen mit der Komplexität besser umgehen als der andere. Das einfache Fallbeispiel wurde gewählt, um sicherzustellen, dass beide Datenbanken ohne Probleme mit dem Datenmodell zurechtkommen. Zudem sollte damit eine gewisse Anzahl an Szenarien der Schema Evolution abgedeckt werden, was die Kernthematik dieser Arbeit ist.

In Bezug auf die Metriken wäre die Nutzung eines Datenbanken Management Systems von Vorteil gewesen, um einen besseren Vergleich mit Neo4j zu garantieren. Durch die Verwendung von SQLite ist der Vergleich der Arbeitsspeichernutzung sowie der LOC irreführend. SQLite ist als eingebettete Datenbank für kleine Anwendung konzipiert, während Neo4j eine vollwertige Server Anwendung ist. Damit ist ein möglichst kleiner Speicherverbrauch für SQLite gewünscht, während Neo4j verschiedene Daten zur weiteren Nutzung im Speicher vorhält. Zudem erlaubt SQLite nicht das nachträgliche Hinzufügen eines Fremdschlüssels zu einer Tabelle. Daher müssen in den Szenarien neue Tabellen erstellt werden, um die Fremdschlüssel anzupassen. Das führt zu zusätzlichen Statements und LOC, die in einer anderen relationalen Datenbank nicht nötig gewesen wären.

5 Fazit

In dieser Arbeit wurden zunächst relationale sowie Graph Datenbanken vorgestellt und ihre Besonderheiten hinsichtlich der Schema Evolution herausgestellt. Zusätzlich wurden weitere Arbeiten aus dem Themenbereich vorgestellt, um ein Bild von den konkreten Vor- und Nachteilen der jeweiligen Datenbanktypen im direkten Vergleich zu erhalten. Anschließend wurde ein beispielhafter Workflow gezeigt, auf Basis dessen einige Szenarien für den darauf folgenden Test erstellt wurden. Nach Durchführung der Tests und Bewertung der Messdaten wurde über die Validität der Analyse diskutiert.

Die Testergebnisse zeigen Ähnlichkeiten zu den vorgestellten Arbeiten auf. Dabei ist im allgemeinen die Abfrage- und Manipulationsgeschwindigkeit der Daten bei der gewählten Graph-Datenbank höher als bei der relationalen Datenbank. Allerdings zeigt sich auch, dass die Graph-

Datenbank ein einen höheren Speicherverbrauch auf der Festplatte wie auch im Arbeitsspeicher besitzt, verglichen mit der relationalen Datenbank.

Insgesamt hat sich gezeigt, dass die genutzte Graph Datenbank Neo4j nur wenige konkrete objektive Vorteile gegenüber der eingesetzten relationalen Datenbank SQLite besitzt in Bezug auf die Schema Evolution des dargestellten Beispielworkflows. Zwar benötigte die Durchführung der Szenarien weniger Code-Arbeit, dafür schneidet Neo4j schlechter hinsichtlich der physikalischen Größe ab.

Damit spricht wenig für einen Wechsel der Datenbank in BPMS. Insbesondere ein Wechsel von einer bestehenden relationalen Datenbank wäre aufgrund zusätzlicher Arbeit und eines wahrscheinlich auftretenden Umgewöhnungsfaktors unwirtschaftlich.

Quellenverzeichnis

- [1] W. van der Aalst und K. M. van Hee, *Workflow management: Models, methods and systems*, 1. MIT Press paperback ed. Cambridge, Mass.: MIT Press, 2004.
- [2] M. Amatayakul, *Process Improvement with Electronic Health Records: A Stepwise Approach to Workflow and Process Management*. Hoboken: CRC Press, 2012.
- [3] Object Management Group, *BPMN Specification - Business Process Model and Notation*, 2019. [Online]. Verfügbar unter: <http://www.bpmn.org/>. [Zugegriffen: 25. Sep 2019].
- [4] M. zur Mühlen, *Workflow-based process controlling: Foundation, design, and application of workflow driven process information systems: Zugl.: Münster (Westfalen), Univ., Diss, 2002*, Bd. 6. Berlin: Logos-Verl., 2004.
- [5] C. A. Curino, H. J. Moon, M. Ham, und C. Zaniolo, *The PRISM Workbench: Database Schema Evolution without Tears*, in *IEEE 25th International Conference on Data Engineering, 2009*, 2009, S. 1523–1526.
- [6] L. C. Rivero, J. H. Doorn, und V. E. Ferragline, Hrsg., *Encyclopedia of Database Technologies and Applications*. IGI Global, 2005.
- [7] H. Balsters, B. Brock, und S. Conrad, Hrsg., *Database Schema Evolution and Meta-Modeling: 9th International Workshop on Foundations of Models and Languages for Data and Objects FoMLaDO/DEMM 2000 Dagstuhl Castle, Germany, September 18-21, 2000 Selected Papers*, Bd. 2065. Berlin, Heidelberg: Springer-Verlag Berlin Heidelberg, 2001.
- [8] Camunda Services GmbH, *The Camunda BPM Workflow Engine*, 2018. [Online]. Verfügbar unter: <https://camunda.com/products/bpmn-engine/>. [Zugegriffen: 25. Sep 2019].
- [9] Camunda Services GmbH, *History and Audit Event Log*, 2015. [Online]. Verfügbar unter: <https://docs.camunda.org/manual/7.5/user-guide/process-engine/history/>. [Zugegriffen: 25. Sep 2019].
- [10] H. Darwen, *An introduction to relational database theory*. Telluride, CO: Ventus Publishing, 2009.
- [11] C. J. Date, *SQL and relational theory: How to write accurate SQL code*, 2nd ed. Sebastopol, CA: O'Reilly, 2012.
- [12] A. Beaulieu, *Learning SQL*, 2. ed. Beijing: O'Reilly, 2009.
- [13] C. Curino, H. Moon, L. Tanca, und C. Zaniolo, *Schema Evolution in Wikipedia - Toward a Web Information System Benchmark.*, in *ICEIS 2008 - Proceedings of the 10th International Conference on Enterprise Information Systems*, 2008, S. 323–332.
- [14] I. Robinson, J. Webber, und E. Eifrem, *Graph Databases, 2nd Edition*. Beijing: O'Reilly Media, Inc, 2015.

- [15] solid IT GmbH, *DB-Engines Ranking - popularity ranking of graph DMBS*, 2019. [Online]. Verfügbar unter: <https://db-engines.com/en/ranking/graph+dbms>. [Zugegriffen: 27. Sep 2019].
- [16] Neo4j, *Chaper 5. Schema - The Neo4j Cypher Manual v3.5*, 2019. [Online]. Verfügbar unter: <https://neo4j.com/docs/cypher-manual/current/schema/>. [Zugegriffen: 30. Sep 2019].
- [17] C. Vicknair, M. Macias, Z. Zhao, X. Nan, Y. Chen, und D. Wilkins, *A comparison of a graph database and a relational database*, in *Proceedings of the 48th Annual Southeast Regional Conference*, 2010, S. 1.
- [18] G. Jaiswal, *Comparative analysis of Relational and Graph databases*, *IOSR Journal of Engineering*, Bd. 3, Nr. 8, S. 25–27, 2013.
- [19] A. Nayak, A. Poriya, und D. Poojary, *Type of NOSQL Databases and its Comparison with Relational Databases*, *International Journal of Applied Information Systems*, Bd. 5, Nr. 4, S. 16–19, 2013.
- [20] M. Owens, *The Definitive Guide to SQLite*. Berkeley, CA: Michael Owens, 2006.

Initialer Aufbau (SQL)

```
1 CREATE TABLE annahme
2 (
3     id INTEGER PRIMARY KEY AUTOINCREMENT,
4     datum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
5 );
6 CREATE TABLE lagerung
7 (
8     id INTEGER PRIMARY KEY AUTOINCREMENT,
9     start REFERENCES annahme(id)
10 );
11 CREATE TABLE sendung
12 (
13     id INTEGER PRIMARY KEY AUTOINCREMENT,
14     datum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
15     lager REFERENCES lagerung(id)
16 );
```

Szenario 2:

```
1 PRAGMA foreign_keys=off;
2 BEGIN TRANSACTION;
3
4 ALTER TABLE sendung RENAME TO sendung_alt;
5
6 CREATE TABLE sendung
7 (
8     id INTEGER PRIMARY KEY AUTOINCREMENT,
9     datum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
10     start REFERENCES annahme(id)
11 );
12
13 INSERT INTO sendung
14 SELECT s.id, s.datum, l.start
15 FROM sendung_alt s
16 JOIN lagerung l ON
17     s.lager = l.id;
18
19 DROP TABLE sendung_alt;
20 DROP TABLE lagerung;
21 COMMIT;
22 PRAGMA foreign_keys=on;
```

Szenario 1:

```
1 DROP TABLE sendung;  
2 DROP TABLE lagerung;  
3 DROP TABLE annahme;
```

Szenario 3:

```
1 PRAGMA foreign_keys=off;  
2 BEGIN TRANSACTION;  
3  
4 CREATE TABLE begutachtung  
5 (  
6     id INTEGER PRIMARY KEY AUTOINCREMENT,  
7     note INTEGER NOT NULL DEFAULT 0, -- neutral  
8     lager REFERENCES lagerung(id)  
9 );  
10  
11 INSERT INTO begutachtung(id, lager)  
12 SELECT id, lager  
13 FROM sendung;  
14  
15 ALTER TABLE sendung RENAME TO sendung_alt;  
16 CREATE TABLE sendung  
17 (  
18     id INTEGER PRIMARY KEY AUTOINCREMENT,  
19     datum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
20     note REFERENCES begutachtung(id)  
21 );  
22  
23 -- Begutachtung hat immer die selbe id wie die sendung.  
24 INSERT INTO sendung(id, datum, note)  
25 SELECT id, datum, id  
26 FROM sendung_alt;  
27  
28 DROP TABLE sendung_alt;  
29  
30 COMMIT;  
31 PRAGMA foreign_keys=on;
```

Szenario 4:

```
1 ALTER TABLE lagerung  
2     ADD stockwerk INTEGER NOT NULL DEFAULT 0; -- DEFAULT erdgeschoss
```

Initialer Aufbau (Cypher)

```
1 CREATE INDEX ON :Annahme(id)
2 CREATE INDEX ON :Lagerung(id)
3 CREATE INDEX ON :Sendung(id)
4 CREATE CONSTRAINT ON (a:Annahme) ASSERT exists(a.datum)
5 CREATE CONSTRAINT ON (s:Sendung) ASSERT exists(s.datum)
```

Szenario 1:

```
1 MATCH (a:Annahme)-[st:Start]->(l:Lagerung)-[e:Ende]->(s:Sendung)
2 DELETE a, l, s, st, e
3
4 DROP INDEX ON :Annahme(id)
5 DROP INDEX ON :Lagerung(id)
6 DROP INDEX ON :Sendung(id)
7 DROP CONSTRAINT ON (a:Annahme) ASSERT exists(a.datum)
8 DROP CONSTRAINT ON (s:Sendung) ASSERT exists(s.datum)
```

Szenario 2:

```
1 MATCH (a:Annahme)-[st:Start]-(l:Lagerung)-[e:Ende]-(s:Sendung)
2 CREATE (a)-[:Durchleitung]->(s)
3 DETACH DELETE l
4
5 DROP INDEX ON :Lagerung(id)
```

Szenario 3:

```
1 MATCH (a:Annahme)-[st:Start]-(l:Lagerung)-[e:Ende]-(s:Sendung)
2 CREATE (a)-[:Start]->(l)-[:Kontrolle]->(b:Begutachtung { id: s.id,
    note: 0 })-[e:Ende]->(s)
3 DELETE st, e
4
5 CREATE INDEX ON :Begutachtung(id)
6 CREATE CONSTRAINT ON (b:Begutachtung) ASSERT exists(b.note)
```

Szenario 4:

```
1 MATCH (l:Lagerung)
2 SET l.stockwerk = 0
3
4 CREATE CONSTRAINT ON (l:Lagerung) ASSERT exists(l.stockwerk)
```